

// JavaJub

Собес в ITK Academy

Java Junior

Вопросы, задачи и подготовка
к первому техническому интервью

Стек

Java • Spring • Spring Boot • Docker • SQL • Git • OAuth

Содержание

01	Про ИТК Academy и формат собеса	3
02	Стек по вакансии	4
03	Java Core & ООП	5
04	Коллекции	7
05	Исключения и многопоточность (база)	8
06	Spring и Spring Boot	9
07	Docker, Git и CI/CD	11
08	Базы данных и SQL	12
09	Безопасность: OAuth и Keycloak	13
10	Практические задачи	14
11	Pet-проект и self-presentation	18
12	План подготовки + чек-лист	19

01 Про ITK Academy и формат собеседа

ITK Academy — это IT-компания из Таганрога, работающая по принципу аутстаффинга. То есть напрямую к ним «джавистом» ты не идёшь: они дообучают, помогают с резюме и портфолио, а затем устраивают на проекты компаний-партнёров. Это значит, что собес у тебя будет двухэтапный.

► Как обычно устроен путь до оффера

Этап	С кем	Что проверяют
1. Знакомство	Менеджер ITK	Мотивация, стек, ожидания, опыт обучения
2. Тех-собес внутри	Техлид/ментор ITK	Java Core, базовые коллекции, ООП, твой pet-проект
3. Подготовка	Ментор	Доработка резюме, дообучение пробелов, моки
4. Собес у клиента	Тех-команда заказчика	Глубокий тех-собес под стек заказчика
5. Оффер	HR заказчика + ITK	Условия, выход на проект

ФИШКА · Главный нюанс

Ты готовишься не к одному собеседу, а к двум разным. Внутри ITK будут смотреть, насколько тебя «продаваемо» можно отправить клиенту. У клиента — насколько ты вписываешься в их стек и команду. Готовиться нужно по широкой базе.

02 Стек по вакансии

ИТК честно расписывает требования к Junior Java на Хабр Карьере. Вот что нужно знать на момент собеседа:

► Обязательный минимум

- Java/Kotlin на уровне core: типы, ООП, коллекции, исключения
- Git: основные команды, работа с ветками
- Spring: основные модули — web, security, data, cloud, test (на уровне «для чего нужны»)
- Spring Boot: стартеры, зачем они нужны, как написать свой
- Docker: контейнер vs образ, базовые команды
- CI/CD в GitLab или GitHub: что это и как пайплайн собирается
- ACID в реляционных БД, индексы — когда применять
- Pet-проект — обязательно

► Будет плюсом

- OAuth 2.0: общие принципы (clients, scopes, grant types)
- Keycloak: если щупал руками — отдельный жирный плюс
- Java 11+ (по описанию вакансии — это базовый минимум)
- REST API: умение спроектировать простой эндпоинт
- PostgreSQL и MySQL — базовые отличия и использование

ВНИМАНИЕ · Что бросается в глаза

Стек для Junior довольно широкий — Spring Security, Cloud, OAuth, Keycloak обычно ждут от Middle. Это значит: глубоко знать всё не нужно, но «своими словами объяснить, что это и зачем» — обязательно.

03 Java Core & ООП

Это база — без неё дальше не пускают. Для Junior спрашивают концепты, а не тонкости JMM.

► Типовые вопросы

- Расскажите про принципы ООП.
→ Инкапсуляция, наследование, полиморфизм, абстракция. На Junior достаточно объяснить «своими словами» с примерами из жизни.
- В чём разница между JDK, JRE и JVM?
→ JVM — виртуальная машина (исполняет байт-код). JRE = JVM + стандартные библиотеки (нужно для запуска). JDK = JRE + инструменты разработки (javac, jdb, jar).
- Java — компилируемый или интерпретируемый язык?
→ И то, и другое. Сначала компилируется в байт-код (.class), потом JVM его интерпретирует и компилирует «горячие» места JIT-компилятором в нативный код.
- Какие есть примитивные типы и их размеры?
→ byte (1), short (2), int (4), long (8), float (4), double (8), char (2), boolean (~1, JVM-зависимо).
- Что такое автобоксинг и распаковка?
→ Автоматическое преобразование примитива в обёртку (int → Integer) и обратно. Имеет цену: создание объекта, возможный NullPointerException при распаковке null.
- Почему String в Java immutable?
→ Безопасность (передача в файлы, БД, сеть), потокобезопасность, кэширование hashCode, работа String Pool.
- Что такое String Pool?
→ Область в heap (раньше — в PermGen), где хранятся уникальные строковые литералы. String s = "hi" использует пул, new String("hi") — нет.
- Разница между == и equals() для строк.
→ == сравнивает ссылки. equals() сравнивает содержимое. Для строк, созданных литералом, == может вернуть true из-за String Pool, но полагаться на это нельзя.
- Что вернёт someObj.equals(null)?
→ false по контракту. equals не должен бросать NPE на null-аргумент.
- Контракт equals/hashCode.
→ 1) Если a.equals(b), то a.hashCode() == b.hashCode(). 2) Если хэшкоды разные — объекты точно не равны. Если переопределяешь один — переопределяй и второй.
- Что произойдёт, если положить объект в HashSet, а потом изменить поле, участвующее в hashCode?
→ Потеряешь объект — найти его через contains() будет уже нельзя. Классическая ошибка с mutable-объектами в коллекциях.
- Разница между checked и unchecked исключениями.
→ Checked наследуются от Exception (но не RuntimeException) — компилятор требует обработки или throws. Unchecked — от RuntimeException и Error — обрабатывать необязательно.
- Назови несколько unchecked исключений.
→ NullPointerException, ArrayIndexOutOfBoundsException, IllegalArgumentException, ClassCastException, NumberFormatException.
- Можно ли в catch ловить Throwable? Стоит ли?
→ Можно, но не стоит — поймаешь и Error (OutOfMemoryError, StackOverflowError), которые обычно нельзя восстанавливать. Лови конкретные исключения.

- Что такое try-with-resources?
→ Конструкция `try (Resource r = ...) {}`, которая автоматически вызовет `r.close()` в `finally`. Ресурс должен реализовывать `AutoCloseable`.
- Можно ли переопределить static-метод?
→ Нет. Static-методы не участвуют в полиморфизме. Если в подклассе будет метод с такой же сигнатурой — это сокрытие (*hiding*), а не переопределение.
- К каким конструкциям применим модификатор `final`?
→ К классу (нельзя наследовать), методу (нельзя переопределить), переменной (нельзя переприсваивать).
- Что такое абстрактный класс? Чем отличается от интерфейса?
→ Абстрактный класс может содержать состояние и реализованные методы, наследуется только один. Интерфейс — контракт, можно реализовать несколько. С Java 8 в интерфейсах появились `default`-методы.
- Расскажи про модификаторы доступа.
→ `private` — только в классе. (`default/package-private`) — в пакете. `protected` — в пакете + наследникам. `public` — везде.

СОВЕТ · Лайфхак

На вопрос про ООП всегда заготовь жизненный пример. Например, инкапсуляция — это банкомат: ты не знаешь, как он внутри хранит деньги, у тебя есть только публичные методы «снять», «положить», «проверить баланс». Запоминается и тебе, и интервьюеру.

04 Коллекции

Спрашивают всегда. HashMap и ArrayList — обязательно. Готовься рассказывать «как устроено».

- Какие основные интерфейсы в Collection Framework?
→ *Collection (List, Set, Queue), Map (отдельно). List — упорядоченный с дубликатами. Set — без дубликатов. Map — пары ключ-значение.*
- Как устроен ArrayList внутри?
→ *Динамический массив. При нехватке места создаёт новый массив в 1.5 раза больше и копирует данные через Arrays.copyOf.*
- ArrayList vs LinkedList — что когда использовать?
→ *ArrayList: быстрый доступ по индексу $O(1)$, быстрая итерация. LinkedList: быстрая вставка/удаление в начале $O(1)$, но поиск $O(n)$. На практике почти всегда выигрывает ArrayList.*
- Как устроен HashMap?
→ *Массив бакетов (Node[]). Бакет выбирается по хэшу ключа: $(n - 1) \& \text{hash}$, где n — размер массива. При коллизии — связный список, а с 8 элементов в бакете и размере таблицы ≥ 64 он превращается в красно-чёрное дерево.*
- Что такое load factor?
→ *Порог заполнения, по умолчанию 0.75. При $\text{size} \geq \text{capacity} * \text{loadFactor}$ происходит resize: новый массив вдвое больше + перехэширование всех элементов.*
- Сложность операций HashMap.
→ *put, get, remove — $O(1)$ в среднем. В худшем случае $O(\log n)$ благодаря дереву (Java 8+).*
- Можно ли в HashMap положить null-ключ или null-значение?
→ *В HashMap — да (один null-ключ хранится в bucket 0). В ConcurrentHashMap — нельзя ни ключ, ни значение.*
- Что будет, если использовать как ключ изменяемый объект и потом его изменить?
→ *Объект «потеряется». hashCode станет другим, и при поиске мы попадём не в тот bucket.*
- HashMap vs TreeMap vs LinkedHashMap.
→ *HashMap — быстрый, без порядка. TreeMap — отсортирован по ключам, $O(\log n)$. LinkedHashMap — сохраняет порядок вставки или access order.*
- HashSet — на основе чего реализован?
→ *На HashMap, где значение — заглушка (PRESENT). Все операции делегируются HashMap.*
- Что такое fail-fast итератор?
→ *Итератор, который кидает ConcurrentModificationException, если коллекция изменена не через сам итератор. Так работают итераторы у HashMap, ArrayList.*
- Как из List сделать неизменяемый?
→ *List.copyOf(list) (Java 10+) или Collections.unmodifiableList(list) — обёртка, которая бросит UnsupportedOperationException на add/remove.*
- В чём разница между List.of(1,2,3) и new ArrayList<>()?
→ *List.of возвращает immutable-список. Любая попытка изменить — UnsupportedOperationException. И в нём нельзя null.*

05 Исключения и многопоточность (база)

Для Junior спрашивают самое начало, без JMM и lock-free алгоритмов.

► Исключения

- Иерархия исключений в Java.
→ *Throwable* → *Error* (системные, не ловим) и *Exception*. От *Exception* → *checked* и *RuntimeException* (*unchecked*).
- Можно ли в одном *catch* ловить несколько типов исключений?
→ Да, *multi-catch* с Java 7: *catch (IOException | SQLException e)*. Переменная при этом *effectively final*.
- Что выполнится в *finally*, если в *try* *return*?
→ *Finally* выполнится перед фактическим возвратом. Если в *finally* тоже *return* — он перебьёт значение из *try* (анти-паттерн).
- Какое исключение лучше — выбросить своё или использовать стандартное?
→ Если ситуация уникальна для домена — своё (наследник *RuntimeException* обычно). Если стандартное (например, *IllegalArgumentException*) подходит — лучше его.

► Многопоточность — база

- Чем процесс отличается от потока?
→ Процесс — единица ОС со своей памятью. Поток — единица исполнения внутри процесса, потоки делят общую память.
- Способы создать поток в Java.
→ *extends Thread*, *implements Runnable*, через *Callable + ExecutorService*, через *CompletableFuture*. На Java 21+ — *Virtual Threads*.
- Что такое *synchronized*?
→ Ключевое слово для захвата монитора объекта. Гарантирует, что в блок одновременно войдёт только один поток. Можно на методе или на блоке кода.
- Зачем нужен *volatile*?
→ Гарантирует видимость изменений переменной между потоками (без кэширования в регистрах). НЕ гарантирует атомарность операций типа *i++*.
- Состояния потока.
→ *NEW* (создан), *RUNNABLE* (готов или выполняется), *BLOCKED* (ждёт монитор), *WAITING* / *TIMED_WAITING* (ждёт сигнал/время), *TERMINATED*.
- Что делает *Thread.sleep()* и *Thread.yield()*?
→ *sleep* — приостанавливает поток на указанное время (не освобождает мониторы). *yield* — намекает планировщику «дай поработать другим», но это лишь подсказка.

06 Spring и Spring Boot

В вакансии ИТК прямо упомянуты Spring web, security, data, cloud, test и Spring Boot стартеры. На Junior достаточно базы.

► Базовые вопросы

- Что такое Spring и зачем он нужен?
→ Фреймворк для упрощения разработки enterprise-приложений на Java. Главные фичи: Inversion of Control (IoC), Dependency Injection (DI), AOP, готовые модули для web, data, security.
- Что такое IoC и DI?
→ IoC — принцип: контейнер сам управляет жизненным циклом объектов. DI — реализация: зависимости в объект внедряются извне (через конструктор / сеттер / поле).
- Какой способ DI предпочтительнее?
→ Constructor injection. Поля можно сделать final, явно видны обязательные зависимости, легко тестировать без Spring-контекста.
- Чем @Component отличается от @Service, @Repository, @Controller?
→ Технически почти ничем — все они помечают бин для регистрации в контексте. Но семантически: @Service — бизнес-логика, @Repository — работа с БД (плюс перевод исключений в DataAccessException), @Controller — веб-слой.
- Что такое @Autowired и @Qualifier?
→ @Autowired — внедрение зависимости. Если бинов несколько — Spring не знает какой выбрать
→ нужен @Qualifier("имяБина") или @Primary на одном из бинов.
- Какие scopes у бинов?
→ singleton (по умолчанию — один на контекст), prototype (новый при каждом запросе), request, session, application, websocket — для веб-приложений.
- Что такое Spring Boot и чем он отличается от Spring?
→ Spring Boot — это Spring + автоконфигурация + встроенный сервер (Tomcat/Jetty) + удобные стартеры. Минимизирует boilerplate. По сути «Spring без боли с XML».
- Что такое стартер в Spring Boot?
→ Готовый набор зависимостей под задачу. Например, spring-boot-starter-web подтянет Spring MVC, Tomcat, Jackson и т.д. Один импорт — и веб-приложение работает.
- Как Spring Boot узнаёт, что и как настраивать?
→ Через @EnableAutoConfiguration (входит в @SpringBootApplication) + @Conditional. Конфигурации перечислены в META-INF/spring/...AutoConfiguration.imports (или старом spring.factories).
- Что делает @SpringBootApplication?
→ Это композиция трёх аннотаций: @Configuration + @EnableAutoConfiguration + @ComponentScan.
- Как сделать REST-эндпоинт?
→ Класс с @RestController, метод с @GetMapping/@PostMapping. Параметры — @PathVariable, @RequestParam, @RequestBody. Возвращаемый объект сериализуется в JSON через Jackson.
- Как обработать исключение в REST-контроллере?
→ @ExceptionHandler в самом контроллере или глобально через @RestControllerAdvice + @ExceptionHandler.
- Что такое Spring Data JPA?
→ Модуль, который позволяет работать с БД через интерфейсы-репозитории. Достаточно унаследоваться от JpaRepository<Entity, ID> — Spring сам сгенерирует реализацию.
- Как Spring создаёт реализацию репозитория, если ты только написал интерфейс?

→ Через прокси, который генерируется в рантайме. Имя метода (*findByEmail*) парсится и превращается в *SQL/JPQL*.

- Что такое `@Transactional`?

→ Аннотация, которая открывает транзакцию перед методом и коммитит после или откатывает при исключении. По умолчанию откат на *RuntimeException* и *Error*, но не на *checked*.

► Про Spring Security и Cloud (на пальцах)

- Что делает Spring Security?

→ Защищает приложение: аутентификация (кто ты) и авторизация (что тебе можно). Работает через цепочку фильтров перед твоим контроллером.

- Что такое Spring Cloud и для чего он?

→ Набор инструментов для микросервисов: *service discovery* (*Eureka*), конфиг-сервер, *API Gateway*, *Circuit Breaker*. На Junior достаточно знать «что это и зачем».

07 Docker, Git и CI/CD

► Docker

- Чем образ (image) отличается от контейнера (container)?
→ Образ — шаблон, неизменяемый снимок (как класс). Контейнер — запущенный экземпляр образа (как объект). Из одного образа можно запустить много контейнеров.
- Что такое Dockerfile?
→ Текстовый файл с инструкциями для сборки образа: FROM (базовый образ), COPY, RUN, CMD/ENTRYPOINT и т.д.
- Какие команды Docker ты знаешь?
→ `docker build`, `docker run`, `docker ps`, `docker logs`, `docker exec`, `docker stop`, `docker rm`, `docker images`, `docker pull/push`.
- Что такое docker-compose?
→ Инструмент для запуска нескольких связанных контейнеров одной командой через `yaml`-файл. Обычно: приложение + БД + Redis.
- В чём разница между CMD и ENTRYPOINT?
→ ENTRYPOINT задаёт основную исполняемую команду. CMD — аргументы по умолчанию. CMD легко переопределить при `docker run`, ENTRYPOINT — сложнее (через `--entrypoint`).

► Git

- Чем merge отличается от rebase?
→ `merge` создаёт коммит-слияние, история ветвится. `rebase` «переносит» коммиты твоей ветки поверх другой, делая историю линейной. `Rebase` в публичных ветках обычно делать не стоит.
- Что делает git stash?
→ Откладывает незакоммиченные изменения «на полку», возвращая рабочую копию к чистому состоянию. Потом можно `git stash pop`.
- Как откатить последний коммит?
→ `git reset --soft HEAD~1` (оставит изменения), `git reset --hard HEAD~1` (выкинет всё), `git revert HEAD` (создаст новый коммит, который отменит предыдущий — безопасно для общих веток).
- Что такое pull request / merge request?
→ Запрос на слияние твоей ветки в основную. Площадка для code review: коллеги смотрят, комментируют, аппрувят.

► CI/CD

- Что такое CI и CD?
→ CI (Continuous Integration) — автоматическая сборка, тесты, статический анализ при каждом `push`. CD (Continuous Delivery/Deployment) — автоматический деплой в среды.
- Что такое pipeline в GitLab/GitHub Actions?
→ Цепочка `job`'ов, выполняемых на `runner`'ах при определённых событиях (`push`, `merge request`). Описывается в `.gitlab-ci.yml` или `.github/workflows/*.yml`.
- Зачем нужны Docker-образы в CI/CD?
→ Чтобы среда сборки была одинаковой на всех машинах. И на проде запускается тот же образ, что был протестирован.

08 Базы данных и SQL

- Что такое ACID?
 - *Atomicity* — транзакция выполняется целиком или не выполняется вовсе. *Consistency* — БД переходит из одного валидного состояния в другое. *Isolation* — параллельные транзакции не мешают друг другу. *Durability* — после `commit` данные не пропадут даже при сбое.
- Уровни изоляции транзакций.
 - `READ_UNCOMMITTED`, `READ_COMMITTED` (default в PostgreSQL), `REPEATABLE_READ`, `SERIALIZABLE`. От низкого к высокому: больше консистентности, меньше параллелизма.
- Что такое dirty read, non-repeatable read, phantom read?
 - *Dirty*: чтение незакоммиченных изменений другой транзакции. *Non-repeatable*: повторное чтение той же строки даёт другой результат. *Phantom*: повторный `SELECT` с `WHERE` возвращает другое количество строк.
- Зачем нужны индексы?
 - Чтобы ускорить поиск. Без индекса — *full scan* $O(n)$. С *B-tree* индексом — $O(\log n)$. Платим за это: дополнительная память + замедление `INSERT/UPDATE/DELETE`.
- Когда индекс не стоит создавать?
 - Маленькие таблицы (*full scan* быстрее). Часто меняющиеся колонки. Колонки с малым количеством уникальных значений (например, `boolean`) — индекс не даст выигрыша.
- Какие виды JOIN ты знаешь?
 - `INNER JOIN` — пересечение. `LEFT JOIN` — все из левой + совпадения. `RIGHT JOIN` — наоборот. `FULL OUTER JOIN` — всё.
- Разница между WHERE и HAVING?
 - `WHERE` фильтрует строки ДО группировки и работает с индексами. `HAVING` — ПОСЛЕ группировки, для условий на агрегаты (`HAVING SUM(amount) > 1000`).
- Чем отличается DELETE от TRUNCATE?
 - `DELETE` — построчное удаление с триггерами и логированием, может быть отквачено в транзакции. `TRUNCATE` — быстрая очистка таблицы целиком, обнуляет автоинкремент.
- Простой запрос: найти пользователей с количеством заказов больше 5.
 - `SELECT user_id, COUNT(*) FROM orders GROUP BY user_id HAVING COUNT(*) > 5;`

09 Безопасность: OAuth и Keycloak

В вакансии ИТК Keycloak отмечен как «преимущество». Глубоко знать не нужно — достаточно общего понимания.

- Что такое OAuth 2.0?
→ Протокол авторизации. Позволяет одному приложению получить доступ к ресурсам пользователя в другом приложении без передачи пароля. Например: «войти через Google».
- Назови основные роли в OAuth.
→ *Resource Owner* (пользователь), *Client* (приложение), *Authorization Server* (выдаёт токены), *Resource Server* (хранит ресурсы).
- Что такое access token и refresh token?
→ *Access token* — короткоживущий, доступ к ресурсам. *Refresh token* — долгоживущий, нужен для получения нового *access token* без повторного входа.
- Чем OAuth отличается от OpenID Connect?
→ OAuth — про авторизацию (что можно). OIDC — надстройка над OAuth для аутентификации (кто ты), добавляет *id_token* с информацией о пользователе.
- Что такое JWT?
→ *JSON Web Token* — компактный формат для передачи информации в виде подписанного JSON. Состоит из трёх частей через точку: *header.payload.signature*. Можно проверять валидность без обращения к серверу.
- Что такое Keycloak?
→ Open-source решение для аутентификации и авторизации. Реализует OAuth 2.0, OIDC, SAML. Поддерживает SSO, federation, social login. Удобен тем, что готовый — не нужно писать свой сервер авторизации.

10 Практические задачи

Для Junior на собеседовании обычно дают что-то из этого списка. Все задачи — реальные форматы с подобных собеседований в аутстафф-компаниях.

► Задача 1. Развернуть строку

Формулировка: напиши метод, который принимает строку и возвращает её в обратном порядке. Без использования `StringBuilder.reverse()`.

```
public String reverse(String input) {
    if (input == null) return null;
    char[] chars = input.toCharArray();
    int left = 0, right = chars.length - 1;
    while (left < right) {
        char tmp = chars[left];
        chars[left] = chars[right];
        chars[right] = tmp;
        left++;
        right--;
    }
    return new String(chars);
}
```

О чём спросят дополнительно:

- А если внутри есть эмодзи или китайские иероглифы (surrogate pairs)? — Там нужен другой подход через `codePoints`.
- Сложность? — $O(n)$ по времени, $O(n)$ по памяти (массив `char`).

► Задача 2. Проверить палиндром

Формулировка: метод `isPalindrome(String s)` возвращает `true`, если строка читается одинаково в обе стороны. Регистр не учитывается, пробелы игнорируются.

```
public boolean isPalindrome(String s) {
    if (s == null) return false;
    int left = 0, right = s.length() - 1;
    while (left < right) {
        while (left < right && !Character.isLetterOrDigit(s.charAt(left))) left++;
        while (left < right && !Character.isLetterOrDigit(s.charAt(right))) right--;
        if (Character.toLowerCase(s.charAt(left)) !=
            Character.toLowerCase(s.charAt(right))) return false;
        left++;
        right--;
    }
    return true;
}
```

► Задача 3. Подсчитать частоту символов

Формулировка: для строки вернуть `Map<Character, Integer>` — сколько раз каждый символ встречается.

```
public Map<Character, Integer> charFrequency(String s) {
    Map<Character, Integer> result = new HashMap<>();
    for (char c : s.toCharArray()) {
        result.merge(c, 1, Integer::sum);
    }
}
```

```
    return result;
}
```

Альтернатива через Stream API:

```
Map<Character, Long> result = s.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(c -> c, Collectors.counting()));
```

► Задача 4. Найти первый неповторяющийся символ

Формулировка: вернуть первый символ строки, который встречается ровно один раз. Если такого нет — вернуть пустой Optional или конкретное значение.

```
public Optional<Character> firstUnique(String s) {
    Map<Character, Integer> counts = new LinkedHashMap<>();
    for (char c : s.toCharArray()) {
        counts.merge(c, 1, Integer::sum);
    }
    return counts.entrySet().stream()
        .filter(e -> e.getValue() == 1)
        .map(Map.Entry::getKey)
        .findFirst();
}
```

ФИШКА · Почему LinkedHashMap, а не HashMap?

LinkedHashMap сохраняет порядок вставки. Если использовать обычный HashMap — порядок ключей не гарантирован, и «первый» неповторяющийся символ может оказаться не тем, что в строке.

► Задача 5. FizzBuzz

Формулировка: вывести числа от 1 до 100. Если число делится на 3 — вывести «Fizz». На 5 — «Buzz». На 15 — «FizzBuzz». Классика.

```
for (int i = 1; i <= 100; i++) {
    if (i % 15 == 0) System.out.println("FizzBuzz");
    else if (i % 3 == 0) System.out.println("Fizz");
    else if (i % 5 == 0) System.out.println("Buzz");
    else System.out.println(i);
}
```

Подвох: проверка % 15 должна идти ПЕРВОЙ. Если поставить её в конце — никогда не сработает.

► Задача 6. Проверить, являются ли две строки анаграммами

Формулировка: две строки — анаграммы, если состоят из одного и того же набора символов. Например: «listen» и «silent».

```
public boolean isAnagram(String a, String b) {
    if (a == null || b == null || a.length() != b.length()) return false;
    char[] aChars = a.toLowerCase().toCharArray();
    char[] bChars = b.toLowerCase().toCharArray();
    Arrays.sort(aChars);
    Arrays.sort(bChars);
    return Arrays.equals(aChars, bChars);
}
```

► Задача 7. Реализовать Singleton

Формулировка: напиши потокобезопасный синглтон. Расскажи, какие есть варианты.

Вариант 1: enum (рекомендуемый)

```
public enum Singleton {
    INSTANCE;

    public void doWork() {
        // ...
    }
}
```

Самый простой и безопасный способ. JVM гарантирует единственность экземпляра, есть защита от рефлексии и сериализации.

Вариант 2: static holder (lazy init)

```
public class Singleton {
    private Singleton() {}

    private static class Holder {
        private static final Singleton INSTANCE = new Singleton();
    }

    public static Singleton getInstance() {
        return Holder.INSTANCE;
    }
}
```

Вариант 3: double-checked locking

```
public class Singleton {
    private static volatile Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

ЛОВУШКА • Зачем volatile?

Без volatile другой поток может увидеть «полусозданный» объект — JVM имеет право переупорядочить операции (выделение памяти → присвоение ссылки → вызов конструктора). Volatile запрещает такие reordering'и.

► Задача 8. Простой REST-эндпоинт на Spring Boot

Формулировка: «Напиши контроллер, который возвращает список пользователей и позволяет добавить нового». Это часто просят сделать прямо во время собеседования.

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping
    public List<UserDto> getAll() {
        return userService.findAll();
    }

    @GetMapping("/{id}")
    public UserDto getById(@PathVariable Long id) {
        return userService.findById(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public UserDto create(@RequestBody @Valid CreateUserRequest request) {
        return userService.create(request);
    }
}
```

На что обратят внимание:

- Constructor injection (а не @Autowired над полем).
- Возвращаешь DTO, а не Entity (защита от утечки внутренних полей).
- Используешь @Valid для валидации входных данных.
- Правильные HTTP-статусы: 200 для GET, 201 для POST.

► Задача 9. Code Review (могут попросить)

Тебе показывают кусок кода и просят найти проблемы. Пример:

```
public class UserCache {
    private static Map<Long, User> cache = new HashMap<>();

    public static User getUser(Long id) {
        if (cache.containsKey(id)) {
            return cache.get(id);
        }
        User user = loadFromDb(id);
        cache.put(id, user);
        return user;
    }
}
```

Что не так

- HashMap не потокобезопасен. В многопоточной среде можно поймать infinite loop (Java 7) или потерять данные.
- Двойной вызов containsKey + get — лишняя операция. Лучше использовать computeIfAbsent.
- Кэш бесконечный — память будет течь. Нужен размер или TTL (Caffeine, Guava Cache).
- Поле public static — глобальное состояние, тяжело тестировать. Лучше сделать spring-бин.
- Нет обработки случая, когда loadFromDb вернул null.

Как должно быть

```
@Service
public class UserCache {
    private final Map<Long, User> cache = new ConcurrentHashMap<>();
    private final UserRepository repository;

    public UserCache(UserRepository repository) {
        this.repository = repository;
    }

    public Optional<User> getUser(Long id) {
        User cached = cache.computeIfAbsent(id,
            key -> repository.findById(key).orElse(null));
        return Optional.ofNullable(cached);
    }
}
```

11 Pet-проект и self-presentation

ИТК прямо требует наличие pet-проекта. Без него тебя «продать» клиенту почти невозможно. Это не просто «галочка» — это твоё портфолио.

► Какой pet-проект подходит

- REST API на Spring Boot с реальной задачей (не «todo-list» — таких миллион). Например: трекер привычек, агрегатор RSS, мини-CRM.
- PostgreSQL или MySQL с понятной схемой и миграциями (Flyway/Liquibase).
- Spring Security + JWT для авторизации.
- Docker Compose, который поднимает приложение + БД одной командой.
- Тесты: хотя бы юнит на сервисный слой + один-два интеграционных через Testcontainers.
- GitHub Actions / GitLab CI: при push прогоняются тесты.
- README с описанием, скриншотами или curl-примерами эндпоинтов.

СОВЕТ · Главное правило

На собеседовании про этот проект ОБЯЗАТЕЛЬНО спросят: что было сложно, почему выбрал такие технологии, что бы переделал сейчас. Готовь ответы заранее. Идеально — иметь пару архитектурных решений, которые ты можешь объяснить и обосновать.

► Что спросят про твой опыт

- Расскажи о себе и почему пошёл в Java.
- Какие курсы заканчивал, что было самым полезным?
- Расскажи про твой pet-проект: как устроен, какие технологии, что было сложно.
- Что читаешь / смотришь по Java? (Назови хотя бы пару источников).
- Готов ли работать удалённо / в офисе / по гибриду?
- Какие зарплатные ожидания?
- Готов ли к командировкам / выходу на проект клиента?

12 План подготовки + чек-лист

► За 2–3 недели до собеседа

1. Прорешать 30 простых задач на LeetCode (Easy) на строки, массивы, мапы. Без алгоритмов на графах — для джуна не нужно.
2. Перечитать главы по коллекциям и ООП в любой хорошей книге (Head First Java, или «Java для начинающих» Сьерра).
3. Поднять локально pet-проект на Spring Boot 3.x с PostgreSQL. Если уже есть — отрефакторить, добавить тесты.
4. Изучить базовые команды Docker и Git (на уровне «делал руками»).

► За неделю

5. Сделать 1–2 мок-собеса (друзья, ChatGPT в роли интервьюера, pramp.com).
6. Прорешать вопросы из этого гайда вслух — вслух, потому что мысли в голове и слова в речи — разные вещи.
7. Подготовить 2–3 истории про опыт по схеме: проблема → что сделал → результат.
8. Прочитать описание вакансии ИТК ещё раз, выписать незнакомые слова — наугулить.

► В день собеседа

- Проверить камеру, микрофон, интернет.
- Открыть IDE, в которой умеешь работать (IntelliJ IDEA Community подойдёт).
- Подготовить блокнот и ручку — рисовать схемы.
- Рассуждать вслух. Тишина для интервьюера хуже, чем «я сейчас подумаю...».
- Не знаешь — скажи: «не сталкивался, но могу предположить, что...» — это лучше, чем тишина.
- В конце задай 2–3 вопроса: про команду, проект, процесс адаптации.

ВНИМАНИЕ · Важно про аутстафф

В ИТК два уровня собеседа. Если завалил у клиента — не катастрофа, тебя предложат другому проекту. Главное — пройти внутренний тех-собес и показать готовность учиться. Ментор в ИТК заинтересован, чтобы ты вышел на проект.

► Финальный чек-лист

Блок	Готов, если можешь...
Java Core	Объяснить ООП-принципы своими словами с примерами
Коллекции	Рассказать, как устроен HashMap, и в чём разница с ArrayList/LinkedList
Исключения	Различать checked и unchecked, объяснить try-with-resources
Многопоточность	Назвать состояния потока и роль synchronized/volatile
Spring	Написать REST-контроллер с DI через конструктор
Spring Boot	Объяснить, что такое стартер и автоконфигурация

Docker	Различить образ и контейнер, написать простой Dockerfile
SQL	Написать запрос с GROUP BY и HAVING, объяснить ACID
Git	Решить merge-конфликт и откатить коммит
OAuth	Объяснить access/refresh token и зачем нужен Keycloak
Pet-проект	Защитить свой проект 5 минут без подсказок

Удачи на собесе!

// git push origin first-offer

JavaJub